Module 4 - Trees

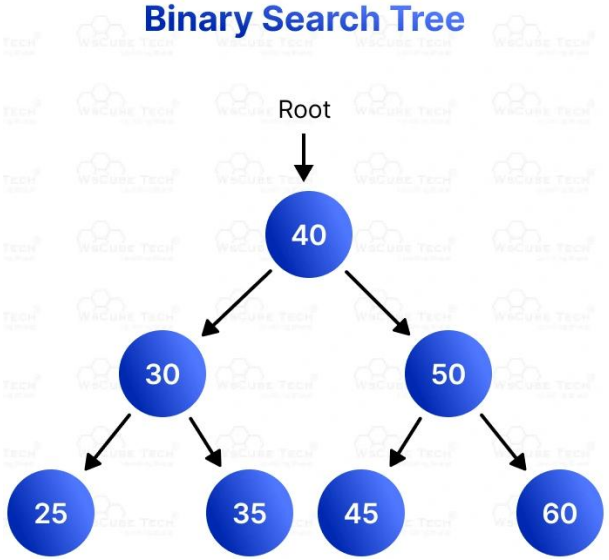
Introduction - Binary Tree: Definition and Properties - Tree Traversals- Expression Trees; Binary Search Trees - Operations in BST: insertion, deletion, finding min and max, finding the kth minimum element.

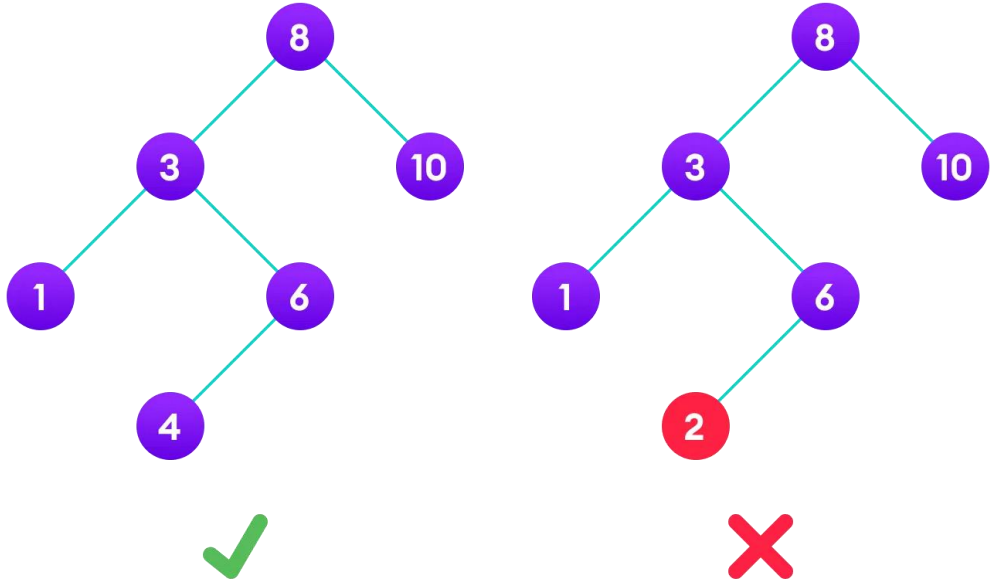
Binary Search Tree

- In a binary search tree (BST) each node is a part of the tree that holds a value. Each node can have up to two children nodes. The node at the top is called the root.

The main rule of a BST is that for any node:

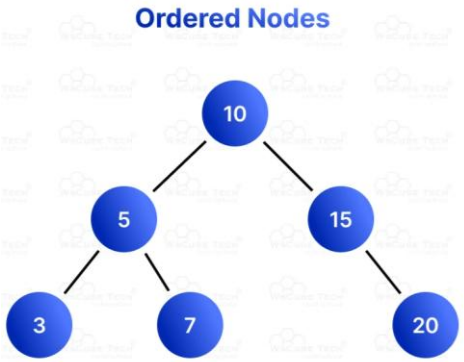
- All the values in the **left child** and its descendants are **smaller than the node's value**.
- All the values in the **right child** and its descendants are **greater than the node's value.**
- This structure helps to quickly find, add, or remove values from the tree.





Binary Search Tree Properties

A Binary Search Tree (BST) has specific properties that make it efficient for **search, insertion, and deletion operations**:



1. Node Structure

- Each node in a BST contains three parts:
- Data: The value stored in the node.
- Left Child: A pointer/reference to the left child node.
- Right Child: A pointer/reference to the right child node.

2. Binary Tree: A BST in data structure is a type of binary tree, which means each node can have at most two children: left and right.

3. Ordered Nodes

- Left Subtree: For any given node, all the values in its left subtree are smaller than the value of the node.
- Right Subtree: For any given node, all the values in its right subtree are greater than the value of the node.

4. No Duplicate Values :A BST in data structure does not contain duplicate values. Each value must be unique to maintain the order property.

5. Recursive Definition : Each subtree of a BST is also a BST. This means the left and right children of any node are roots of their own Binary Search Trees.

Binary Search Tree Operations

1. Insertion

Insertion in binary search tree includes adding a new node in a way that maintains the BST property: for any node, all values in its left subtree are smaller, and all values in its right subtree are greater.

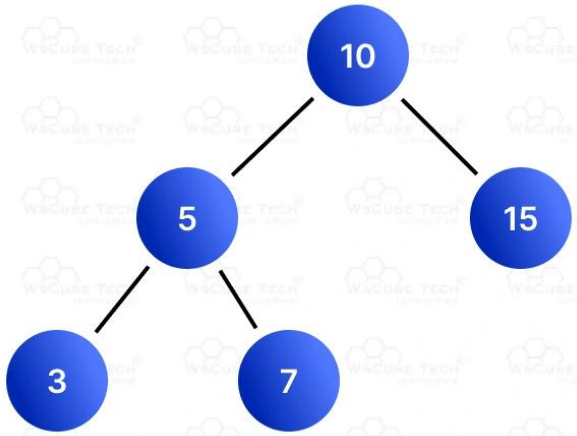
Algorithm:

1. Start at the root node.
2. Compare the value to be inserted with the current node's value.
3. If the value is smaller, move to the left child.
4. If the value is greater, move to the right child.
5. Repeat steps 2-4 until you find an empty spot.
6. Insert the new node at the empty spot.

Insertion Example

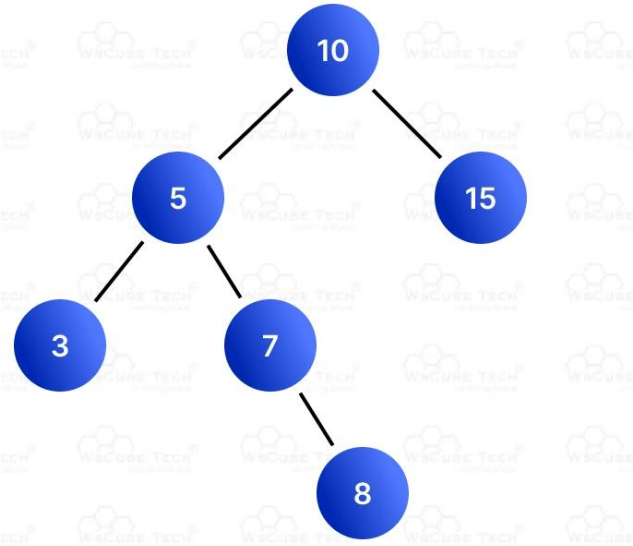
1. Insertion

- **Example:** Insert the value 8 into the following BST:



Step-by-Step Insertion:

1. Start at the root (10).
2. 8 is less than 10, move to the left child (5).
3. 8 is greater than 5, move to the right child (7).
4. 8 is greater than 7, move to the right child of 7 (which is empty).
5. Insert 8 as the right child of 7.



Deletion

- Binary search tree node deletion involves removing a node and reorganizing the tree to maintain the BST property.

There are three cases to handle:

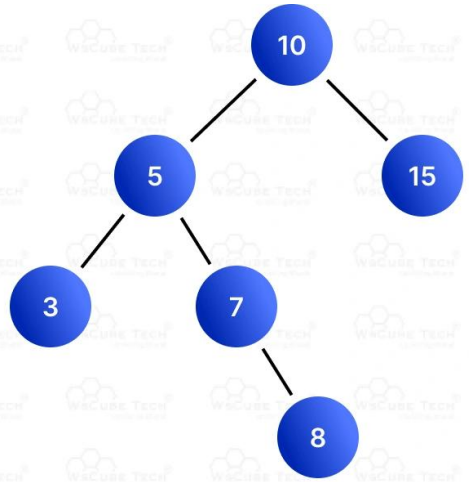
1. **Node with no children (leaf node)**
2. **Node with one child**
3. **Node with two children**

Algorithm:

1. **No Child (Leaf Node):** Simply remove the node.
2. **One Child:** Remove the node and replace it with its child.
3. **Two Children:** Find the in-order predecessor (largest value in the left subtree) or in-order successor (smallest value in the right subtree), replace the node's value with the predecessor/successor, and delete the predecessor/successor.

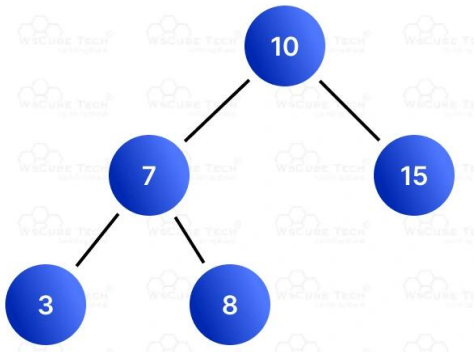
Deletion Example

Delete the value 5 from the following BST:



Step-by-Step Deletion:

1. Start at the root (10).
2. 5 is less than 10, move to the left child (5).
3. Node 5 has two children. Find the in-order successor (smallest value in the right subtree of 5), which is 7.
4. Replace 5 with 7.
5. Delete node 7.



Searching in BST

Searching in a binary search tree involves finding a node with a given value by leveraging the BST property to reduce the number of comparisons.

Algorithm:

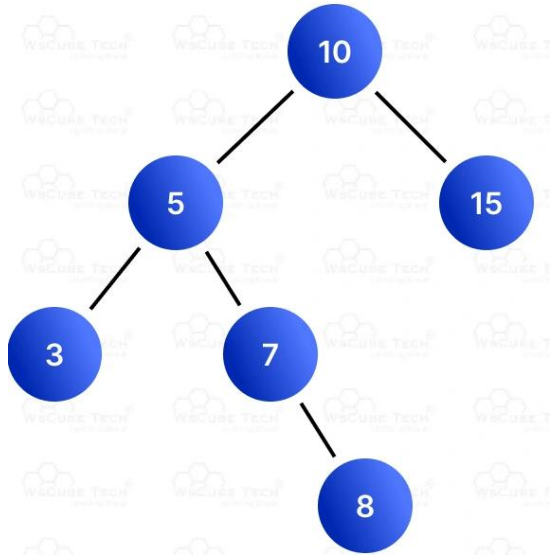
1. Start at the root node.
2. Compare the target value with the current node's value.
3. If the value is equal, the search is successful.
4. If the value is smaller, move to the left child.
5. If the value is greater, move to the right child.
6. Repeat steps 2-5 until you find the value or reach an empty spot (unsuccessful search).

Example Searching in BST

Search for the value 8 in the following BST:

Step-by-Step Searching:

1. Start at the root (10).
2. 8 is less than 10, move to the left child (5).
3. 8 is greater than 5, move to the right child (7).
4. 8 is greater than 7, move to the right child (8).
5. 8 is equal to 8, search is successful.

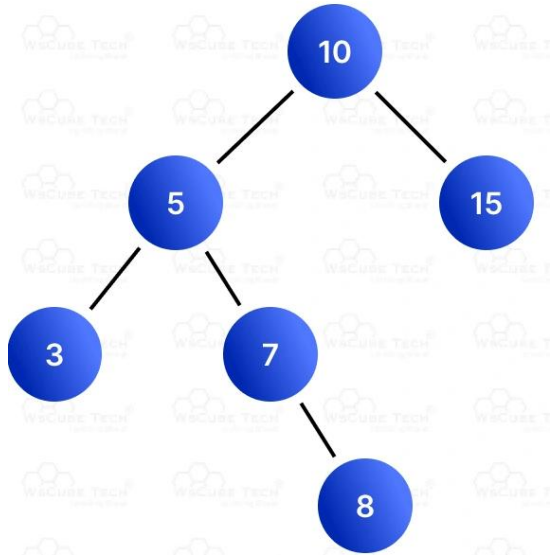


Example Searching in BST

Search for the value 8 in the following BST:

Step-by-Step Searching:

1. Start at the root (10).
2. 8 is less than 10, move to the left child (5).
3. 8 is greater than 5, move to the right child (7).
4. 8 is greater than 7, move to the right child (8).
5. 8 is equal to 8, search is successful.



Construction of unique Binary tree

The construction of a Binary Search Tree (BST) from traversal orders **requires at least two traversals to uniquely define the tree.**

Method 1: Using Inorder and Preorder traversals

Preorder traversal order: Root -> Left -> Right

Inorder traversal order: Left -> Root -> Right

Algorithm:

1. **Identify the root:** The first element of the preorder traversal is the root of the tree.
2. **Find root in inorder:** Search for this root value in the inorder traversal. All elements to its left form the left subtree, and all elements to its right form the right subtree.
3. **Divide traversals:** Based on the position of the root in the inorder array, partition both the preorder and inorder arrays into left and right subtree sections.
4. **Recursively build subtrees:**

- The next elements in the preorder sequence correspond to the root of the left subtree.
- The remaining elements in preorder correspond to the root of the right subtree.
- Make recursive calls to build the left and right subtrees.

5. **Return**: Return the constructed root node

Construction of unique Binary tree

Example:

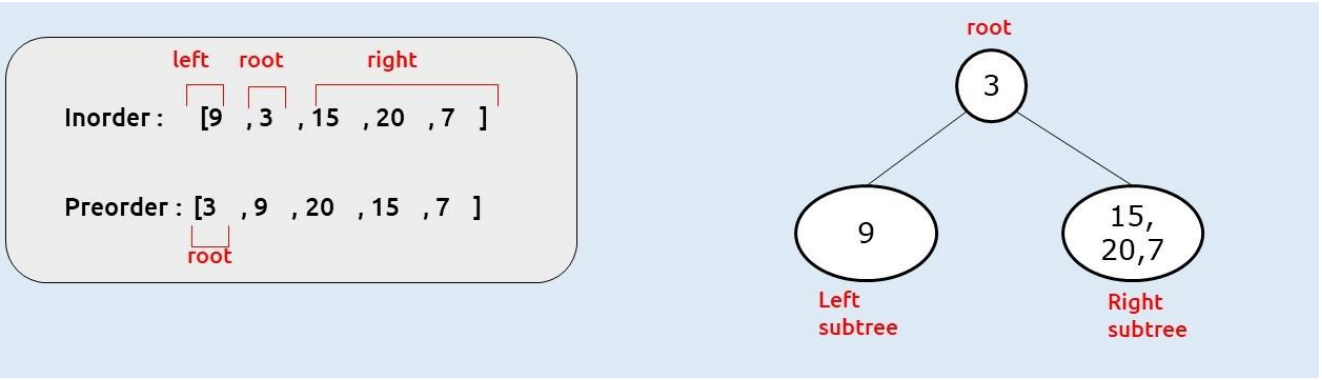
- **Inorder**: [40, 20, 50, 10, 60, 30]
- **Preorder**: [10, 20, 40, 50, 30, 60]

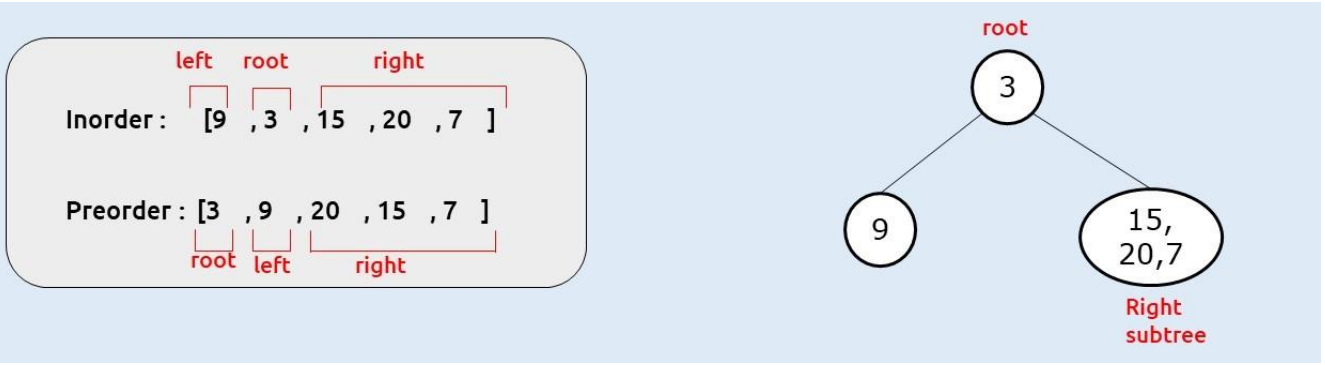
Steps:

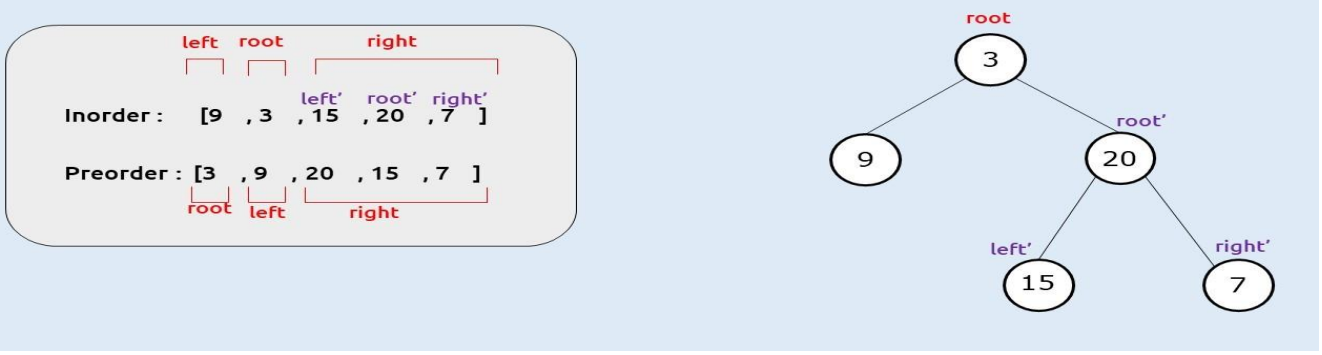
1. The root is 10 (first in preorder).
2. In the inorder list, 10 is at index 3.
3. The left subtree's elements are [40, 20, 50]. The right subtree's elements are [60, 30].
4. Recursively build the left subtree with preorder [20, 40, 50] and inorder [40, 20, 50].
5. Recursively build the right subtree with preorder [30, 60] and inorder [60, 30]

Construction of unique Binary tree

Example:







Given

Inorder : [9, 3, 15, 20, 7]

Preorder : [3, 9, 20, 15, 7]

Construction of unique Binary tree

Method 2: Using Inorder and Postorder traversals

Postorder traversal order: Left -> Right -> Root

Inorder traversal order: Left -> Root -> Right

Algorithm:

1. Identify the root: The last element of the postorder traversal is the root of the tree.
2. Find root in inorder: Find this root value in the inorder traversal to partition the elements into left and right subtrees.
3. Recursively build subtrees:
    I. Because postorder processes the root last, work backwards through the postorder array. The element just before the current root belongs to the right subtree.
    II. Make a recursive call for the right subtree, then for the left subtree.
4. Return: Return the constructed root node

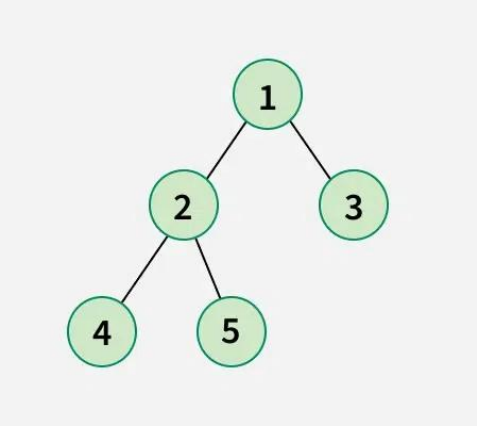
Construction of unique Binary tree

Example:

- In-order: [4, 2, 5, 1, 3]
- Post-order: [4, 5, 2, 3, 1]

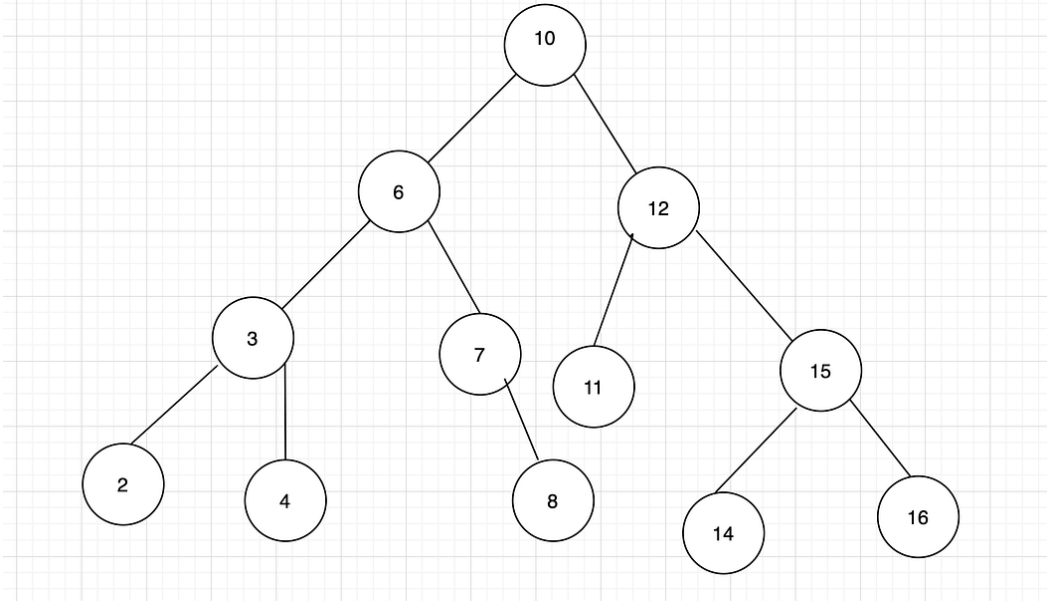
Steps:

1. The root is 1 (last in post-order).
2. In the in-order list, 1 is at index 3.
3. The left subtree's elements are [4, 2, 5]. The right subtree's elements are [3].
4. The last element of the left subtree post-order is 2. Recursively build the left subtree.
5. The last element of the right subtree post-order is 3. Recursively build the right subtree



Finding Max/Min in BST

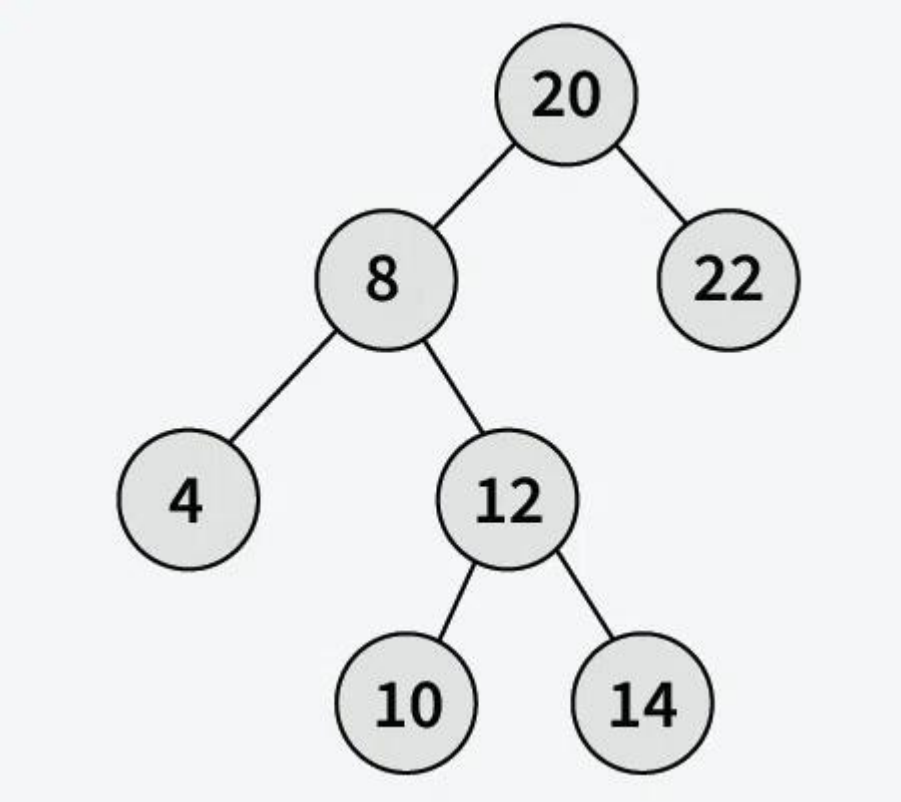
```
function max(rootNode){
    let maxElement = rootNode
    if(rootNode === null){
        return maxElement
    }
    if(rootNode.right === null){
      return maxElement
    }
    if(rootNode.right.data > maxElement.data){
        maxElement = rootNode.right
        return max(rootNode.right)
    }
}
 function min(rootNode){
     let minElement = rootNode
     if(rootNode === null){
         return minElement
     }
     if(rootNode.left === null){
         return minElement
     }
     if(rootNode.left.data < minElement.data){
         minElement = rootNode.left
         return min(rootNode.left)
     }
 }
```

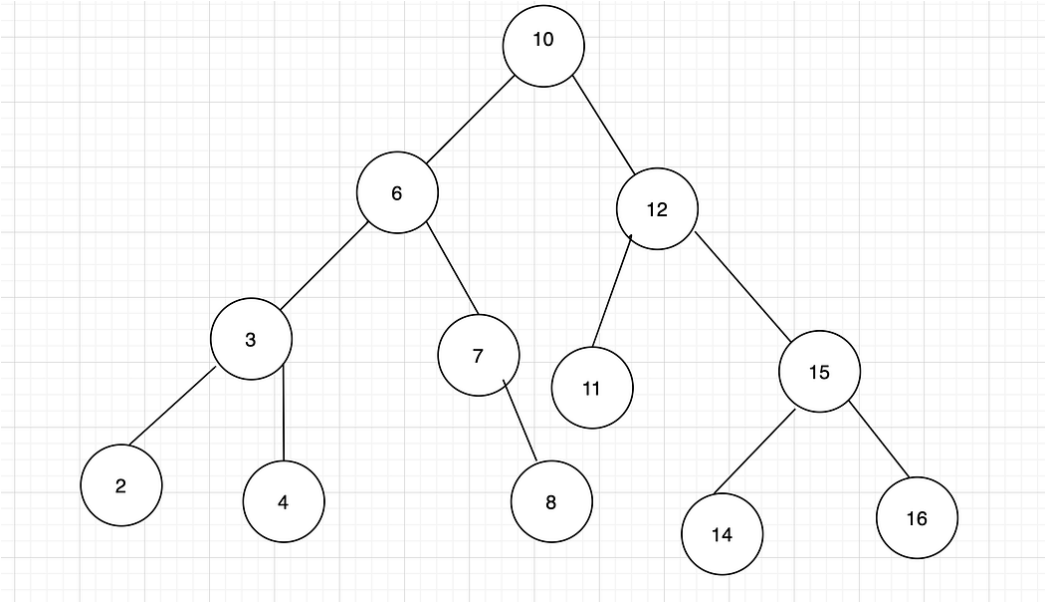


Finding Kth largest element in BST

- Given a Binary Search Tree (BST) and a positive integer k, find the kth smallest element in the Binary Search Tree.

Explanation: The inorder trave rsal of given BST is [4, 8, 10, 12, 14, 20, 22] and its 3rd smallest element is 10.





the index of the Kth smallest element would be K-1 and the index of the Kth largest element would be 1-K or (size of elements) - K.

Finding Kth smallest element in BST

Given a Binary Search Tree (BST) and a positive integer k, find the kth smallest element in the Binary Search Tree.

Explanation: The inorder traversal of given BST is [4, 8, 10, 12, 14, 20, 22] and its 3rd smallest element is 10.